

MBZUAI

AI1215 — Introduction to Machine Learning

CreditSense: Loan Risk Assessment Challenge

Final Project Report

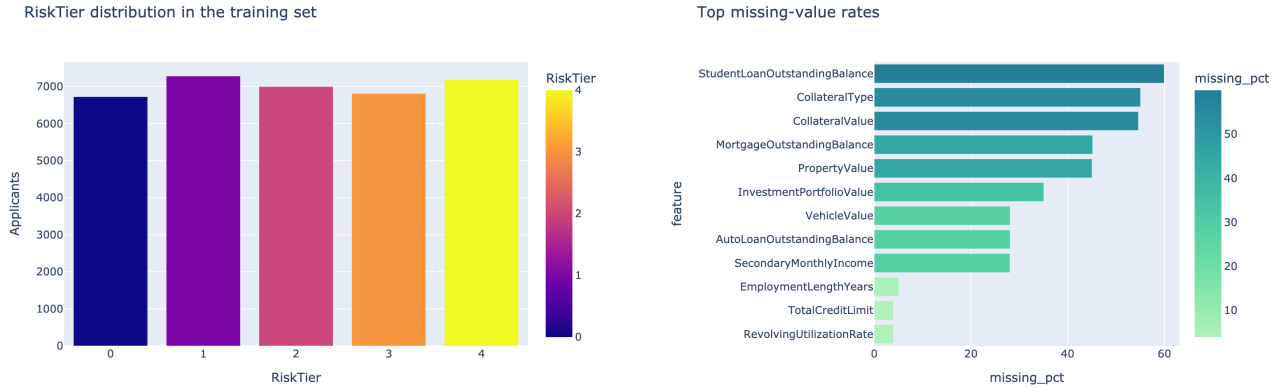
Team name:	Johnny-Izhak-Kyriakos
Students:	Johnny, Izhak, Kyriakos
Kaggle username:	JohnnyC0rp
Final submission family:	meta_overnight
Date:	April 2026

This report summarizes the final submitted pipeline for the CreditSense challenge. The task combines a five-class classification problem with a bounded regression problem, and Kaggle scores each submission by averaging classification accuracy and regression R^2 . Our development therefore focused on changes that improved both targets at the same time, while keeping the final solution reproducible from the saved branch artifacts and the final implementation.

1 Data Exploration and Preprocessing

The training set contains 35,000 applicants, 55 predictors, and two targets: `RiskTier` for Task A and `InterestRate` for Task B. Because the public score is the mean of accuracy and R^2 , every design choice was checked against both targets on the same fixed validation split. This was important in practice: a change that helped classification but damaged the rate model was usually not worth keeping.

The first useful observation was that the dataset was not dominated by class imbalance. Figure 1 shows that all five risk tiers are well represented, so we did not use class reweighting or resampling. The more serious issue was structured missingness. The columns with the highest missing rates were `StudentLoanOutstandingBalance` (59.98%), `CollateralType` (55.06%), `CollateralValue` (54.64%), `MortgageOutstandingBalance` (45.18%), and `PropertyValue` (45.05%). Those gaps were not random; they reflected borrower state.



(a) Risk tiers are reasonably balanced.

(b) Missingness is concentrated in meaningful financial columns.

Figure 1: The early data check was reassuring in one way and tricky in another: class balance was fine, but missing values clearly carried signal.

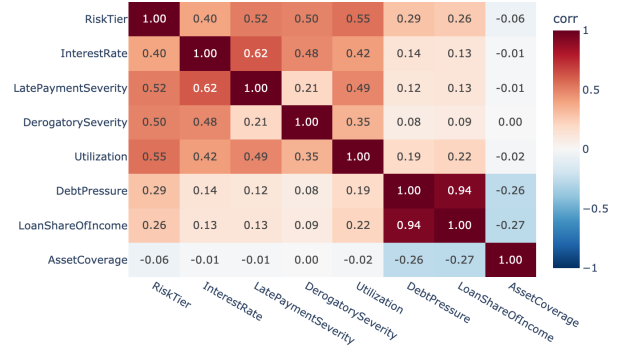
That point becomes clearer once the missing columns are conditioned on context. In Figure 2, `PropertyValue` and `MortgageOutstandingBalance` are missing for about 80% of renters and applicants living with family, but only for about 16–17% of owners or mortgage holders. Student-loan balances show a similar pattern by age: they are absent for the entire oldest age bucket and present much more often for younger borrowers. We therefore treated missingness as a feature to explain, not noise to erase.

Property and mortgage missingness follows home-ownership status



(a) Property and mortgage blanks are tightly linked to home-ownership status.

Selected correlations among the targets and strongest engineered signals



(b) Delinquency and utilization features align most strongly with both targets.

Figure 2: Two findings shaped the rest of the pipeline: missing values were contextual, and the strongest raw relationships were concentrated in delinquency and utilization signals.

The second useful observation was that the two targets overlap, but not perfectly. The mean offered APR rises from 6.01% in tier 0 to 11.55% in tier 4, yet there is still visible overlap across the lower four classes. This is why the regression problem could not simply be treated as a deterministic mapping from the class label. The correlation heatmap in Figure 2 makes the same point from another angle: `RevolvingUtilizationRate`, `LatePaymentSeverity`, and `DerogatorySeverity` correlate most strongly with both targets, while affordability and coverage features are weaker but still complementary.

Preprocessing was intentionally conservative:

- numeric columns were median-filled only after explicit missingness indicators were created;
- categorical columns were preserved as strings for CatBoost, while the shared matrix used stable ordinal encoding and simple frequency features;
- no resampling was applied because the class histogram was already balanced enough;
- one fixed stratified split with seed 42 was reused for branch comparison and feature screening.

These choices kept the raw borrower structure intact. In particular, the preprocessing stage did not try to smooth away financial asymmetries that later became useful engineered features.

2 Feature Engineering

Most of the score jump came from feature work rather than from replacing one learner with another. The original columns already described income, debt, assets, and credit history, but many of the strongest relationships only became clear after the variables were expressed as lending ratios, delinquency summaries, or context-aware flags.

Feature family	Examples	Why it stayed in the final submitted pipeline
Affordability ratios	loan share of income, monthly payment share, asset coverage	These features turned raw balances into stress measures that were interpretable for both tasks.
Delinquency summaries	weighted late-payment counts, late payments per history year, utilization interactions	They were the most reliable features for separating higher risk tiers and explaining large APR jumps.
Derogatory burden	charge-off, collection, bankruptcy, and public-record aggregates	A compact summary of past credit damage that avoided over-relying on any single raw count.
Semantic missingness	renter-without-property, owner-without-mortgage, no-collateral flags	These features captured why a field was absent, which often mattered more than the filled value.
Controlled interactions	selected pairwise products between stress and balance features	A small interaction set gave the tree models non-linear structure without exploding the feature space.

Table 1: Feature groups retained in the final submitted pipeline.

Table 2 tracks the large additions that moved the validation score during the project. The first major step was the jump from the assignment baseline to the ratio-and-missingness feature set. Controlled polynomial terms gave only a small extra gain, which was a useful warning that feature count alone was not the answer. The next substantial jump came from combining different tree families on top of the engineered table. Later gains from target statistics and the neural branch were real, but smaller.

Stage	Combined score	Gain vs previous	Main effect
Assignment baseline	0.5100	—	Starter notebook with minimal preprocessing.
Domain ratios and semantic missingness	0.8225	+0.3125	Added affordability, utilization, and context-aware missingness features.
Controlled polynomial interactions	0.8251	+0.0026	Added a restricted interaction set around stress and debt variables.
Tree ensemble diversification	0.8495	+0.0243	Combined XGBoost, LightGBM, CatBoost, Extra-Trees, and the MPS branch.
Target statistics and bagging	0.8557	+0.0062	Improved categorical summaries for later local experiments.
Neural branch and weighted fusion	0.8576	+0.0019	Added a smoother neural error profile and a stronger late-stage combiner.

Table 2: How major feature and model additions changed the validation score.

Figure 3 shows the target-specific ranking of the strongest engineered variables. The two tasks clearly overlap, but they do not react to exactly the same form of signal. For classification, the most useful variables are raw delinquency counts, utilization, and their interaction terms. For regression, the ordering leans even more strongly toward severity interactions such as `LateSeverityXUtilization`. In other words, the same borrower history matters for both tasks, but the rate model is slightly more sensitive to how several risk signals combine.

Top engineered features for classification vs regression

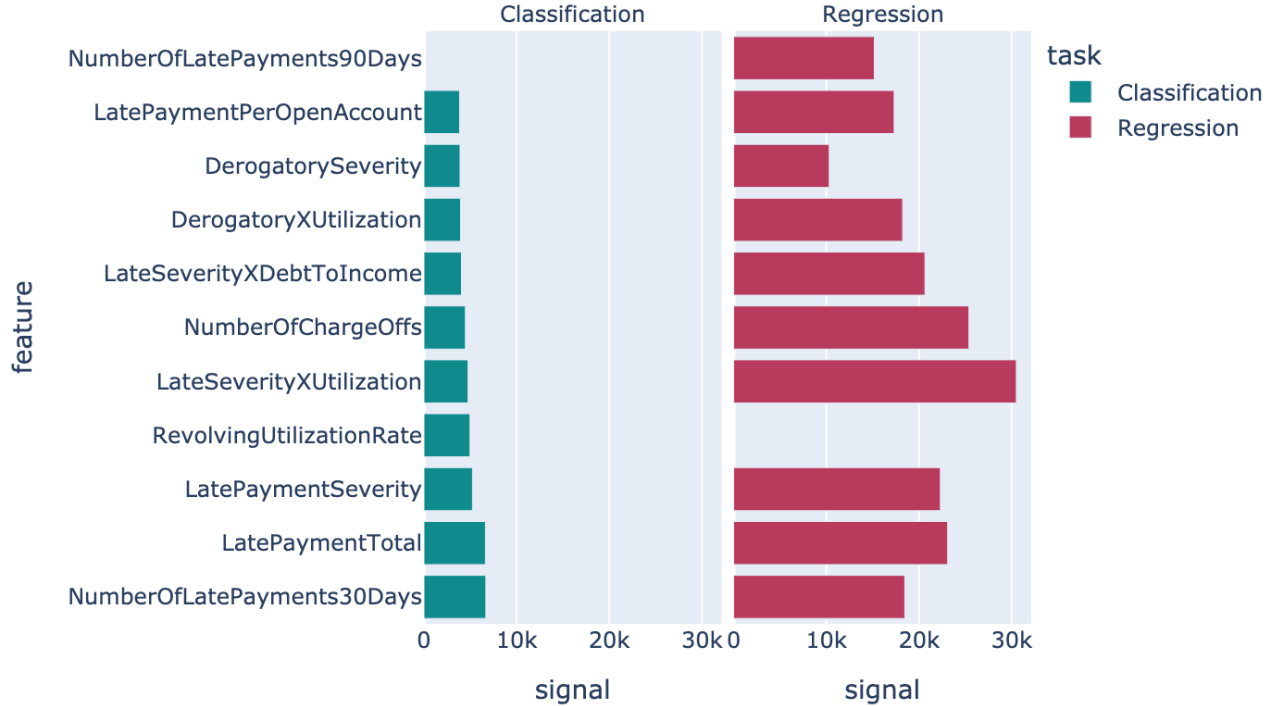


Figure 3: The strongest engineered features overlap across targets, but the ordering is not identical.

The final feature set was therefore not a maximal one. We kept only feature groups that either improved both tasks directly or increased diversity for the ensemble stage.

3 Model Selection & Final Pipeline

Once the feature set stopped changing, model selection became easier to organize. The branch search followed one simple rule: keep model families only when they added either stronger single-model performance or genuinely different errors. The final submission combined five branches:

1. XGBoost on the shared engineered numeric matrix;
2. LightGBM on the same matrix, with a different boosting bias;
3. CatBoost on the mixed table with categorical columns preserved as categories;
4. ExtraTrees as a weaker but diverse non-boosted tree baseline;
5. a compact MPS neural branch trained jointly for classification and regression.

Table 3 summarizes the main saved checkpoints from this search. Among the single branches, LightGBM and CatBoost gave the most balanced results, while the MPS branch was useful because its mistakes were less aligned with the tree branches. ExtraTrees was clearly weaker on its own, but keeping one simpler branch still helped the combined models by adding diversity. The best submitted solution was not the weighted average but the meta-ensemble, which stacked branch outputs with a logistic classifier for Task A and a ridge regressor for Task B.

Model	Accuracy	R^2	Combined	Comment
XGBoost	0.7953	0.8453	0.8203	Strong regressor after feature engineering, weaker classifier than the best boosters.
LightGBM	0.8063	0.8440	0.8251	Best balanced single tree branch on the shared matrix.
CatBoost	0.8027	0.8478	0.8253	Strongest single regressor; benefited from raw categorical columns.
ExtraTrees	0.6934	0.7305	0.7120	Weak alone, but still added diversity inside the ensemble.
MPS neural branch	0.8334	0.8059	0.8197	Different error profile; more useful in combination than alone.
Weighted blend	0.8424	0.8485	0.8455	Simple averaging already beat every single branch.
Meta ensemble	0.8477	0.8513	0.8495	Final submitted pipeline and best saved submission-ready score.

Table 3: Model families tried for the final submission lane and their validation results.

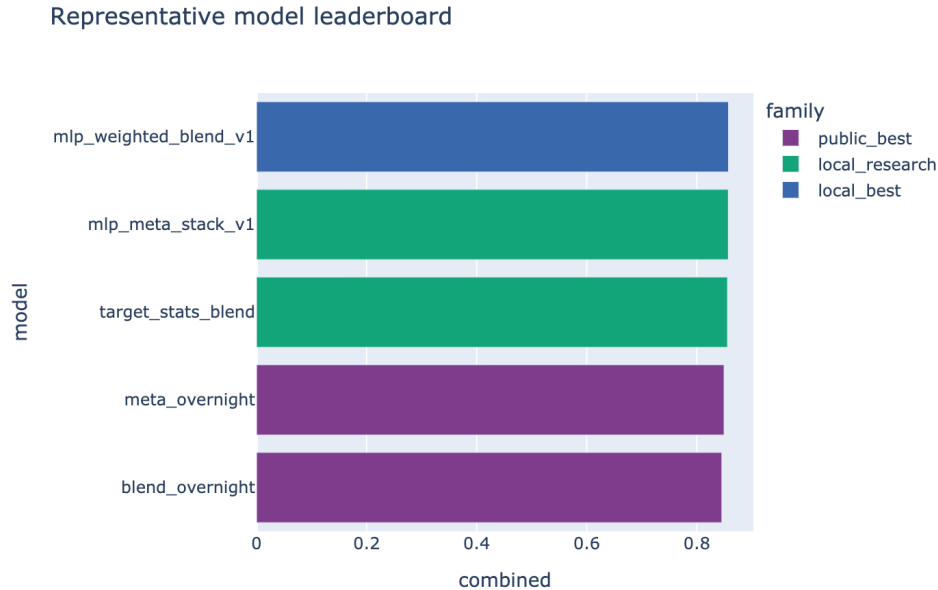


Figure 4: Leaderboard view of the saved model checkpoints used in the main report.

Overfitting control. Every branch was compared on the same stratified split, the interaction set was kept deliberately small, and the tree models were tuned under fixed time budgets before refitting only the selected settings. The MPS branch added dropout, weight decay, gradient clipping, and patience-based early stopping, while the meta-ensemble tuned its logistic and ridge combiners through cross-validation rather than an unrestricted second-level learner.

Do the same features matter equally for both tasks? Broadly yes, but not in exactly the same form. Both targets depend on delinquency history, derogatory marks, and utilization; however, classification reacts more directly to raw counts, whereas regression leans more strongly on interaction terms such as `LateSeverityXUtilization`. This is why every branch kept separate classification and regression heads until the final submission stage.

Final submitted pipeline. Inference proceeds in five steps: build the shared engineered table; train XGBoost, LightGBM, CatBoost, ExtraTrees, and the multitask MPS branch on the saved overnight settings; collect branch probabilities and rate predictions; fit the logistic and ridge meta-combiners; then clip the final APR to $[4.99, 35.99]$ and round it to two decimals.

This `meta_overnight` lane reached a combined local validation score of 0.8495. Later local research branches pushed the score higher, but the submitted model remained the cleanest fully specified pipeline for the assignment deliverable.